

Programming Language Principles, Design, and Implementation

Compilers - Code Optimization

Vincent Rahli

University of Birmingham

Where are we?

- ▶ Part 1: Principles of programming
- ▶ **Part 2: Compilers**

Today

Code Optimization

Today

Code Optimization

- ▶ Local Optimizations
- ▶ Local Common Subexpressions Elimination
- ▶ Dead Code Elimination
- ▶ Arithmetic Transformations
- ▶ Copy Propagation
- ▶ Redundant Loads/Stores
- ▶ Flow-of-Control Optimizations
- ▶ Global Optimizations

Today

Code Optimization

- ▶ Local Optimizations
- ▶ Local Common Subexpressions Elimination
- ▶ Dead Code Elimination
- ▶ Arithmetic Transformations
- ▶ Copy Propagation
- ▶ Redundant Loads/Stores
- ▶ Flow-of-Control Optimizations
- ▶ Global Optimizations

Further reading:

- ▶ Chapter 8 & 9 of the Dragon book
- ▶ Part II of “Modern Compiler Implementation”

Problem

Code generated by a compiler can often be improved

- ▶ the code generated **by the programmer** can be improved
- ▶ the code generate **by the compiler**'s transformations can be improved, e.g., because it contains redundancies

Problem

Code generated by a compiler can often be improved

- ▶ the code generated **by the programmer** can be improved
- ▶ the code generated **by the compiler's** transformations can be improved, e.g., because it contains redundancies

Example

Tackling the programmer:

```
1 if (x < n + m) { ... }  
2 if (y < n + m) { ... }
```

Tackling the compiler:

$$t1 = x$$
$$t2 = x$$

Problem

Code generated by a compiler can often be improved

- ▶ the code generated **by the programmer** can be improved
- ▶ the code generated **by the compiler's** transformations can be improved, e.g., because it contains redundancies

Example

Tackling the programmer:

Tackling the compiler:

```
1 if (x < n + m) { ... }  
2 if (y < n + m) { ... }
```

$t1 = x$

$t2 = x$

Compilers often include optimization phases:

- ▶ **machine-independent code optimizations**
- ▶ **machine-dependent code optimizations**

Problem

The word “optimization” can be misleading: the problem of generating **optimal** code from a given source program is **undecidable**.

Problem

The word “optimization” can be misleading: the problem of generating **optimal** code from a given source program is **undecidable**.

Optimization phases are critical parts of a compiler

- ▶ they need to **preserve the semantics** of programs
- ▶ should generate **efficient code**

Problem

The word “optimization” can be misleading: the problem of generating **optimal** code from a given source program is **undecidable**.

Optimization phases are critical parts of a compiler

- ▶ they need to **preserve the semantics** of programs
- ▶ should generate **efficient code**

Optimization phases can for example optimize:

- ▶ **runtime**
- ▶ **memory usage**
- ▶ **power consumption**
- ▶ ...

Problem

The word “optimization” can be misleading: the problem of generating **optimal** code from a given source program is **undecidable**.

Optimization phases are critical parts of a compiler

- ▶ they need to **preserve the semantics** of programs
- ▶ should generate **efficient code**

Optimization phases can for example optimize:

- ▶ **runtime**
- ▶ **memory usage**
- ▶ **power consumption**
- ▶ ...

Optimizations can be done at the level of **flow graphs**

- ▶ **local**: within a basic block
- ▶ **global**: across basic blocks

Local Optimizations

Examples of local optimizations:

- ▶ elimination of **local common subexpressions**: computations done multiple times
- ▶ elimination of **dead code**: useless computations
- ▶ reordering of statements (e.g., to limit the time a value is stored in a register)
- ▶ reordering of operands of three-address instructions (e.g., arithmetic operations to simplify computations)

Local Optimizations

Examples of local optimizations:

- ▶ elimination of **local common subexpressions**: computations done multiple times
- ▶ elimination of **dead code**: useless computations
- ▶ reordering of statements (e.g., to limit the time a value is stored in a register)
- ▶ reordering of operands of three-address instructions (e.g., arithmetic operations to simplify computations)

To help with some local optimizations, we first convert a basic block into a **DAG** as follows:

- ▶ the initial values of variables are nodes
- ▶ for each statement, we create a node
 - ▶ whose children are the nodes corresponding to the statements that last defined the operands
 - ▶ labeled by the operator of the statement
 - ▶ annotated with the variables for which it is the last definition

Local Common Subexpressions

Example: the following basic block

$$a = b + c$$

$$b = a - d$$

$$c = b + c$$

$$d = a - d$$

Local Common Subexpressions

Example: the following basic block

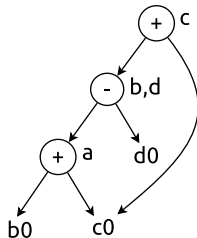
$$a = b + c$$

$$b = a - d$$

$$c = b + c$$

$$d = a - d$$

can be converted into:



Note: when adding a node for $d = a - d$, we notice that there is already a node with the same operator and children

Local Common Subexpressions

After we have performed some optimizations using a DAG, we must **reconstitute a TAC**:

- ▶ we generate a TAC statement for each node (respecting the order of nodes in the DAG)
- ▶ for each node that has more than one variable attached to it, we have to generate **copy** statements (unless it is not further used).

Local Common Subexpressions

After we have performed some optimizations using a DAG, we must **reconstitute a TAC**:

- ▶ we generate a TAC statement for each node (respecting the order of nodes in the DAG)
- ▶ for each node that has more than one variable attached to it, we have to generate **copy** statements (unless it is not further used).

When reconstructing code from the above DAG, we can generate:

$$a = b + c$$

$$b = a - d$$

$$d = b$$

$$c = b + c$$

Local Common Subexpressions

After we have performed some optimizations using a DAG, we must **reconstitute a TAC**:

- ▶ we generate a TAC statement for each node (respecting the order of nodes in the DAG)
- ▶ for each node that has more than one variable attached to it, we have to generate **copy** statements (unless it is not further used).

When reconstructing code from the above DAG, we can generate:

$$a = b + c$$

$$b = a - d$$

$$d = b$$

$$c = b + c$$

and if b is not used in another block, we can even generate

$$a = b + c$$

$$d = a - d$$

$$c = d + c$$

Dead Code Elimination

A variable is **live** if it is used in another block.

Dead Code Elimination

A variable is **live** if it is used in another block.

To remove dead code, we repeat the following operation:

- ▶ remove roots not annotated with live variables

Dead Code Elimination

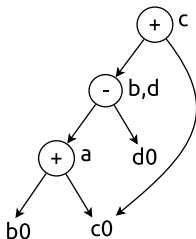
A variable is **live** if it is used in another block.

To remove dead code, we repeat the following operation:

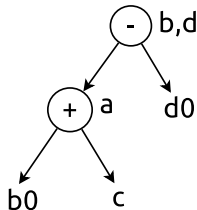
- ▶ remove roots not annotated with live variables

For example, if c is not live

we transform:



into:



Dead Code Elimination

Convert the following TAC into a DAG and perform dead code elimination assuming that e and c are not live

$$a = b + c$$

$$b = b - d$$

$$c = c + d$$

$$e = b + c$$

Dead Code Elimination

Convert the following TAC into a DAG and perform dead code elimination assuming that e and c are not live

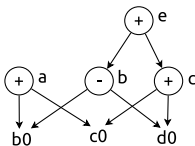
$$a = b + c$$

$$b = b - d$$

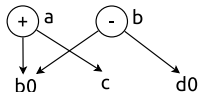
$$c = c + d$$

$$e = b + c$$

DAG:



after dead state elimination:



Arithmetic Transformations

The compiler can also make use of **algebraic laws** to optimize the code, such as

- ▶ identity laws of $+$: $x + 0 = x$ and $0 + x = x$
- ▶ identity laws of $\times$: $1 \times x = x$ and $x \times 1 = x$
- ▶ identity law of $-$: $x - 0 = x$
- ▶ identity law of $/$: $x/1 = x$

Arithmetic Transformations

The compiler can also make use of **algebraic laws** to optimize the code, such as

- ▶ identity laws of $+$: $x + 0 = x$ and $0 + x = x$
- ▶ identity laws of $\times$: $1 \times x = x$ and $x \times 1 = x$
- ▶ identity law of $-$: $x - 0 = x$
- ▶ identity law of $/$: $x/1 = x$

Some arithmetic operations might be **more expansive than others**:

- ▶ $x^2 = x \times x$
- ▶ $2 \times x = x + x$

Arithmetic Transformations

The compiler can also make use of **algebraic laws** to optimize the code, such as

- ▶ identity laws of $+$: $x + 0 = x$ and $0 + x = x$
- ▶ identity laws of $\times$: $1 \times x = x$ and $x \times 1 = x$
- ▶ identity law of $-$: $x - 0 = x$
- ▶ identity law of $/$: $x/1 = x$

Some arithmetic operations might be **more expansive than others**:

- ▶ $x^2 = x \times x$
- ▶ $2 \times x = x + x$

Some computations involving constants only can be performed at compile time (**constant folding**):

- ▶ $2 * 4$ can be replaced by 8

Arithmetic Transformations

The compiler can also make use of **algebraic laws** to optimize the code, such as

- ▶ identity laws of $+$: $x + 0 = x$ and $0 + x = x$
- ▶ identity laws of $\times$: $1 \times x = x$ and $x \times 1 = x$
- ▶ identity law of $-$: $x - 0 = x$
- ▶ identity law of $/$: $x/1 = x$

Some arithmetic operations might be **more expansive than others**:

- ▶ $x^2 = x \times x$
- ▶ $2 \times x = x + x$

Some computations involving constants only can be performed at compile time (**constant folding**):

- ▶ $2 * 4$ can be replaced by 8

Further laws can sometimes be used, e.g., associativity and commutativity of $+$ and $\times$

Arithmetic Transformations

Such transformations can have a compounded effect:

Arithmetic Transformations

Such transformations can have a compounded effect:

Apply any of the above transformations to

$$a = b + c$$

$$b = d + d$$

$$c = b + c$$

$$d = 2 * d$$

Arithmetic Transformations

Such transformations can have a compounded effect:

Apply any of the above transformations to

$$a = b + c$$

$$b = d + d$$

$$c = b + c$$

$$d = 2 * d$$

We can apply 1 arithmetic transformation:

$$a = b + c$$

$$b = d + d$$

$$c = b + c$$

$$d = d + d$$

Arithmetic Transformations

Such transformations can have a compounded effect:

Apply any of the above transformations to

$$a = b + c$$

$$b = d + d$$

$$c = b + c$$

$$d = 2 * d$$

We can apply 1 arithmetic transformation:

$$a = b + c$$

$$b = d + d$$

$$c = b + c$$

$$d = d + d$$

Followed by a common subexpressions elimination transformation:

$$a = b + c$$

$$b = d + d$$

$$c = b + c$$

$$d = b$$

Copy Propagation

Some transformations, such as common subexpressions elimination, introduce **copies** that can be **propagated**, and then eliminated using dead-code elimination.

Copy Propagation

Some transformations, such as common subexpressions elimination, introduce **copies** that can be **propagated**, and then eliminated using dead-code elimination.

Given a copy statement of the form $a = b$, as long as a and b do not get re-assigned, we can replace a with b .

Copy Propagation

Some transformations, such as common subexpressions elimination, introduce **copies** that can be **propagated**, and then eliminated using dead-code elimination.

Given a copy statement of the form $a = b$, as long as a and b do not get re-assigned, we can replace a with b .

For example:

$$a = d$$

$$b = a - d$$

$$c = b + c$$

$$d = a - d$$

Copy Propagation

Some transformations, such as common subexpressions elimination, introduce **copies** that can be **propagated**, and then eliminated using dead-code elimination.

Given a copy statement of the form $a = b$, as long as a and b do not get re-assigned, we can replace a with b .

For example:

$$\begin{aligned}a &= d \\b &= a - d \\c &= b + c \\d &= a - d\end{aligned}$$

Using **copy propagation**:

$$\begin{aligned}a &= d \\b &= d - d \\c &= b + c \\d &= d - d\end{aligned}$$

Copy Propagation

Some transformations, such as common subexpressions elimination, introduce **copies** that can be **propagated**, and then eliminated using dead-code elimination.

Given a copy statement of the form $a = b$, as long as a and b do not get re-assigned, we can replace a with b .

For example:

$$\begin{aligned}a &= d \\b &= a - d \\c &= b + c \\d &= a - d\end{aligned}$$

Using **copy propagation**:

$$\begin{aligned}a &= d \\b &= d - d \\c &= b + c \\d &= d - d\end{aligned}$$

Using **dead code elimination**, if a is not used in another block:

$$\begin{aligned}b &= d - d \\c &= b + c \\d &= d - d\end{aligned}$$

Elimination of Redundant Loads/Stores

The code can be optimized at multiple level. Here we see an example of an optimization at **assembly level**.

Elimination of Redundant Loads/Stores

The code can be optimized at multiple level. Here we see an example of an optimization at **assembly level**.

If we end up in the following situation where a **store** instruction directly follows a **load** instruction

LD *R0*, *a*

ST *a*, *R0*

then the **store** instruction can be safely **removed**.

Elimination of Redundant Loads/Stores

The code can be optimized at multiple level. Here we see an example of an optimization at **assembly level**.

If we end up in the following situation where a **store** instruction directly follows a **load** instruction

```
LD R0, a
ST a, R0
```

then the **store** instruction can be safely **removed**.

Can we also remove the store instruction in:

```
LD R0, a
L : ST a, R0
```


Elimination of Redundant Loads/Stores

The code can be optimized at multiple level. Here we see an example of an optimization at **assembly level**.

If we end up in the following situation where a **store** instruction directly follows a **load** instruction

```
LD R0, a
ST a, R0
```

then the **store** instruction can be safely **removed**.

Can we also remove the store instruction in:

```
LD R0, a
L : ST a, R0
```

No:

- ▶ In the first example without labels, we were guaranteed that by the time we execute the **store** instruction, *a* would be in register *R0*
- ▶ In the second example with a label, this is not guaranteed: another variable might be in *R0*

Flow-of-Control Optimizations

A code generator might introduce **unnecessary jumps**:

- ▶ jumps to jumps
- ▶ jumps to conditional jumps
- ▶ conditional jumps to jumps

Flow-of-Control Optimizations

A code generator might introduce **unnecessary jumps**:

- ▶ jumps to jumps
- ▶ jumps to conditional jumps
- ▶ conditional jumps to jumps

For example,

the following **jump to jump**:

```
        goto  $L_1$   
        ...  
 $L_1$  : goto  $L_2$ 
```

can be replaced by:

```
        goto  $L_2$   
        ...  
 $L_1$  : goto  $L_2$ 
```

Flow-of-Control Optimizations

A code generator might introduce **unnecessary jumps**:

- ▶ jumps to jumps
- ▶ jumps to conditional jumps
- ▶ conditional jumps to jumps

For example,

the following **jump to jump**:

can be replaced by:

```
        goto  $L_1$   
        ...  
 $L_1$  : goto  $L_2$ 
```

```
        goto  $L_2$   
        ...  
 $L_1$  : goto  $L_2$ 
```

Furthermore, if

- ▶ there are no further jumps to L_1
- ▶ L_1 : goto L_2 is preceded by an unconditional jump

then this can further be optimized to:

```
        goto  $L_2$   
        ...
```

Global Optimizations

Several of these local optimizations can also be performed **globally**, i.e., across basic blocks.

Global Optimizations

Several of these local optimizations can also be performed **globally**, i.e., across basic blocks.

Global optimizations often depend on **data-flow analysis**: deriving information about the flow of data along program execution paths in a **flow graph**.

Global Optimizations

Several of these local optimizations can also be performed **globally**, i.e., across basic blocks.

Global optimizations often depend on **data-flow analysis**: deriving information about the flow of data along program execution paths in a **flow graph**.

A data-flow analysis analyzes the **state** of a program **before and after** each statement, where the state can vary depending on the analysis done. For example, a state can be a set of definitions (assignments of values to variables) when computing the definitions that can reach a given point in the program

Global Optimizations

Several of these local optimizations can also be performed **globally**, i.e., across basic blocks.

Global optimizations often depend on **data-flow analysis**: deriving information about the flow of data along program execution paths in a **flow graph**.

A data-flow analysis analyzes the **state** of a program **before and after** each statement, where the state can vary depending on the analysis done. For example, a state can be a set of definitions (assignments of values to variables) when computing the definitions that can reach a given point in the program

A **data-flow value** is an abstraction of the state before/after a statement. For each statement s , associate

- ▶ $IN[s]$ the data-flow values **before** s
- ▶ $OUT[s]$ the data-flow values **after** s

Global Optimizations

Several of these local optimizations can also be performed **globally**, i.e., across basic blocks.

Global optimizations often depend on **data-flow analysis**: deriving information about the flow of data along program execution paths in a **flow graph**.

A data-flow analysis analyzes the **state** of a program **before and after** each statement, where the state can vary depending on the analysis done. For example, a state can be a set of definitions (assignments of values to variables) when computing the definitions that can reach a given point in the program

A **data-flow value** is an abstraction of the state before/after a statement. For each statement s , associate

- ▶ $IN[s]$ the data-flow values **before** s
- ▶ $OUT[s]$ the data-flow values **after** s

The **data-flow problem** is to find a solution to constraints on all $IN[s]$ and $OUT[s]$

Global Optimizations

Data-flow values are constrained as follows (for *forward-flow problems*):

- ▶ **transfer functions**: given a statement s ,

$$\text{OUT}[s] = f_s(\text{IN}[s])$$

where f_s captures the semantics of s

- ▶ **control-flow constraints**:

- ▶ if block B consists of the sequence of statements $s_1, \dots, s_n$, then

$$\forall i \in \{1, \dots, n-1\}. \text{OUT}[s_i] = \text{IN}[s_{i+1}]$$

- ▶ we write $\text{IN}[B]$ for $\text{IN}[s_1]$ and $\text{OUT}[B]$ for $\text{OUT}[s_n]$
- ▶ $\text{IN}[B] = \bigcup_{B' \text{ is a predecessor of } B} \text{OUT}[B']$

Global Optimizations

Example—Reaching Definitions: find in a program where each variable may have been defined. This allows for example to know whether a variable is a constant for constant folding.

Global Optimizations

Example—Reaching Definitions: find in a program where each variable may have been defined. This allows for example to know whether a variable is a constant for constant folding.

A definition d **reaches** a variable x if there is a path in the flow graph from d to x such that x is not redefined along that path (we say that the definition is **killed**).

Global Optimizations

Example—Reaching Definitions: find in a program where each variable may have been defined. This allows for example to know whether a variable is a constant for constant folding.

A definition d **reaches** a variable x if there is a path in the flow graph from d to x such that x is not redefined along that path (we say that the definition is **killed**).

Fact: deciding whether each path in a flow graph can be taken in an **undecidable problem**. Therefore, computing reaching definitions is done **conservatively**: we assume that all paths can be followed.

Global Optimizations

Example—Reaching Definitions: find in a program where each variable may have been defined. This allows for example to know whether a variable is a constant for constant folding.

A definition d **reaches** a variable x if there is a path in the flow graph from d to x such that x is not redefined along that path (we say that the definition is **killed**).

Fact: deciding whether each path in a flow graph can be taken in an **undecidable problem**. Therefore, computing reaching definitions is done **conservatively**: we assume that all paths can be followed.

Given a definition d of the form (or any other similar assignment):

$$d : u = v + w$$

let f_d be defined as follows (x is a data-flow value, here a definition set):

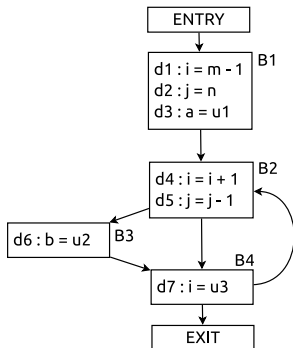
$$f_d(x) = \{d\} \cup (x \setminus \text{kill}_d)$$

where kill_d is the set of all other definitions of u in the program.

Global Optimizations

Example—continued:

initially $\text{IN}[B] = \emptyset$ and $\text{OUT}[B] = \emptyset$

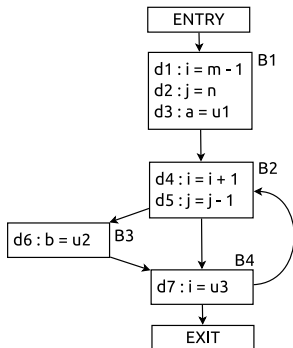


Global Optimizations

Example—continued:

initially $IN[B] = \emptyset$ and $OUT[B] = \emptyset$

Find $IN[B]$ and $OUT[B]$ for all blocks.

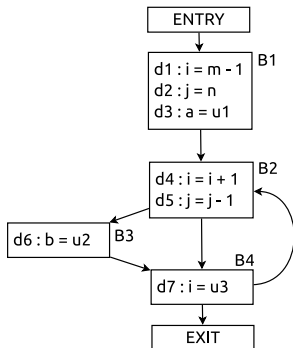


Global Optimizations

Example—continued:

initially $IN[B] = \emptyset$ and $OUT[B] = \emptyset$

Find $IN[B]$ and $OUT[B]$ for all blocks.



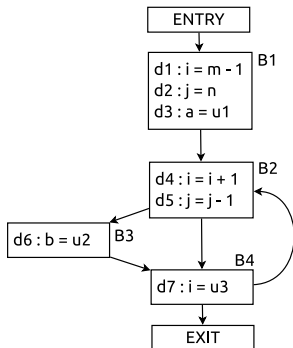
- ▶ $IN[B_1] = \emptyset$ and $OUT[B_1] = \{d_1, d_2, d_3\}$
- ▶ $IN[B_2] = \{d_1, d_2, d_3, d_5, d_6, d_7\}$ and $OUT[B_2] = \{d_3, d_4, d_5, d_6\}$
- ▶ $IN[B_3] = \{d_3, d_4, d_5, d_6\}$ and $OUT[B_3] = \{d_3, d_4, d_5, d_6\}$
- ▶ $IN[B_4] = \{d_3, d_4, d_5, d_6\}$ and $OUT[B_4] = \{d_3, d_5, d_6, d_7\}$

Global Optimizations

Example—continued:

initially $IN[B] = \emptyset$ and $OUT[B] = \emptyset$

Find $IN[B]$ and $OUT[B]$ for all blocks.



- ▶ $IN[B_1] = \emptyset$ and $OUT[B_1] = \{d_1, d_2, d_3\}$
- ▶ $IN[B_2] = \{d_1, d_2, d_3, d_5, d_6, d_7\}$ and $OUT[B_2] = \{d_3, d_4, d_5, d_6\}$
- ▶ $IN[B_3] = \{d_3, d_4, d_5, d_6\}$ and $OUT[B_3] = \{d_3, d_4, d_5, d_6\}$
- ▶ $IN[B_4] = \{d_3, d_4, d_5, d_6\}$ and $OUT[B_4] = \{d_3, d_5, d_6, d_7\}$

In the “exit” of the data flow, if u_1 is a constant, we know that a is a constant too

Global Optimizations

Iterative algorithm:

- ▶ start with $\text{OUT}[\text{ENTRY}] = \emptyset$ and $\text{OUT}[B] = \emptyset$ for all blocks B
- ▶ while one $\text{OUT}[B]$ changes:
 - ▶ for each block B :
 - ▶ $\text{IN}[B] = \bigcup_{B' \text{ is a predecessor of } B} \text{OUT}[B']$
 - ▶ $\text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] \setminus \text{kill}_B)$

where

- ▶ gen_B : latest definitions of variables in B
- ▶ kill_B : union of all kill_d for all definitions d in B

Conclusion

What did we cover today?

- ▶ Local Optimizations
- ▶ Local Common Subexpressions Elimination
- ▶ Dead Code Elimination
- ▶ Arithmetic Transformations
- ▶ Copy Propagation
- ▶ Redundant Loads/Stores
- ▶ Flow-of-Control Optimizations
- ▶ Global Optimizations

Conclusion

What did we cover today?

- ▶ Local Optimizations
- ▶ Local Common Subexpressions Elimination
- ▶ Dead Code Elimination
- ▶ Arithmetic Transformations
- ▶ Copy Propagation
- ▶ Redundant Loads/Stores
- ▶ Flow-of-Control Optimizations
- ▶ Global Optimizations

Next time?

- ▶ Compilation of functional programming languages